a19482b1-0a47-4656-8852-7f8ed5731c

<current>

<request>

You are working on a task in the 'camunda/camunda' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/camunda/camunda (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-04-21T19:59:52.354+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>
Research task (do NOT create or modify any files, do NOT open a pull request):

Investigate and explain comprehensively how **messages** work end-to-end within Zeebe/Camunda, and how **partition routing** influences message handling. The goal is to produce a detailed technical explanation enriched with concrete code snippets (with file paths and line numbers / GitHub permalinks) from the `camunda/camunda` repository.

Please cover the following areas in depth:

1. Message Publishing
- How are messages published to Zeebe (gateway API, broker client, command processors)?
- Trace the path from `PublishMessage` gRPC/REST request through the gateway to the broker.
- Where is the `PublishMessageRequest` defined and how is it transformed into an internal command?
- How does the gateway decide which partition to route the publish-message command to? (This is where hash-based routing on `correlationKey` should appear.)
- Show the actual `PublishMessageProcessor` (or equivalent) on the broker side and what state it writes (MESSAGE.PUBLISHED, etc.).

```
- How is message TTL handled, and how are messages stored in
state (MessageState)?
- How is message deduplication handled via `messageId`?

## 2. Message Subscriptions
Explain the two kinds of subscriptions:
- **Process Message Subscription** (subscription opened by a
process instance waiting on a message catch event / receive
task / message boundary event / message start event).
- **Message Subscription** (the broker-side subscription
tracking which partition is interested in which message name +
correlation key).

For each:
- Where are they opened/closed in the engine?
- What records/intents are involved (e.g.
`MESSAGE_SUBSCRIPTION`, `PROCESS_MESSAGE_SUBSCRIPTION` with
intents like CREATING, CREATE, CREATED, CORRELATING, CORRELATED,
DELETING, DELETED)?
- Show the relevant processors (e.g.
`OpenMessageSubscriptionProcessor`,
`CorrelateMessageSubscriptionProcessor`, etc.) with snippets.
- How do subscriptions get sent across partitions
(SubscriptionCommandSender / SubscriptionCommandMessageHandler)?

## 3. Message Correlation
- How are published messages correlated to waiting
subscriptions?
- Show the correlation flow when a message is published: lookup
of subscriptions, sending correlate commands cross-partition,
ack flow back.
- Show the correlation flow when a subscription is opened and a
buffered message already exists.
- How are message start events handled (they don't have a
process instance yet)?
- How is the correlation key extracted (FEEL expression
evaluation) at both the publish side and the subscribe side?

## 4. Partitioning &amp;amp; Routing
This is the central piece the user asked about. Explain:
- The hash-based partitioning of messages by `correlationKey`.
Where is this implemented? (Look for something like
`SubscriptionUtil`, `getSubscriptionPartitionId`,
`getSubscriptionHashCode`, `RoundRobinDispatchStrategy`,
`RoutingState`, `MessageCorrelationKeyContext`, etc.)
- Show the actual hash function and modulo-by-partition-count
logic.
```

```
- Why hash-based: so that the same correlation key always lands
on the same partition, allowing local correlation between
MessageSubscription and Message state.
- How does the gateway use this to route `PublishMessage`
commands?
- How does the engine on partition A use this to send
`OpenMessageSubscription` commands to the correct partition B
that owns that correlation key?
- How does Zeebe's dynamic scaling / `RoutingState` (if present)
affect this — i.e. how is the number of partitions determined
for hashing? Is there a stable `messageCorrelationPartitions`
value distinct from total partitions?
- Show snippets from any "routing" related files:
`RoutingState`, `RoutingInfo`, `MessageCorrelationPartitions`,
`PartitionScalingChecker`, etc.

## 5. Cross-Partition Communication
- How do partitions talk to each other for subscription open/
correlate/reject/close flows?
- Show the `SubscriptionCommandSender` and the corresponding
receiver/handler.
- Explain the retry / at-least-once semantics for these inter-
partition messages (e.g. `PendingMessageSubscriptionState`,
`PendingProcessMessageSubscriptionState` and the checkers that
resend pending commands).

## 6. Message Buffering &amp;amp; Expiry
- How are messages buffered when no subscription exists yet?
- How does TTL expiration work (`MessageTimeToLiveChecker` or
similar)?
- How are correlated messages cleaned up?

## 7. Message Start Events
- Special handling: how are messages correlated to message start
events to create a new process instance?
- How does deduplication by `messageId` work for start events?

## 8. Message Correlation API (newer feature, if present)
- Is there a synchronous "correlate message" command/endpoint
that waits for correlation? Show how it works and how it
interacts with routing.

## Deliverable Format
For each section above, include:
- A clear narrative explanation.
- Concrete code snippets in fenced code blocks with `name=` and
`url=` headers pointing to the file at the latest main branch
```

(or a specific commit) on GitHub, with line anchors (e.g. `#L10-L40`).
- A short call-graph or sequence summary (text) where helpful.

Focus especially on the **hash-based routing of messages by correlation key** and how it ties together publishing, subscriptions, and correlation across partitions — that is the core of the user's question.

Provide a thorough, well-cited analysis. Reference specific files and line numbers.
</problem_statement>

First, determine whether the <problem_statement> requires code changes, an explanation, a plan, or analysis. Then follow the appropriate path:

If the task requires code changes:

If the task is a question or only requires creating a plan or analysis: respond directly without making code changes.

<prompt>

Investigate and explain comprehensively how **messages** work end-to-end within Zeebe/Camunda, and how **partition routing** influences message handling. The goal is to produce a detailed technical explanation enriched with concrete code snippets (with file paths and line numbers / GitHub permalinks) from the `camunda/camunda` repository.

Please cover the following areas in depth:

1. Message Publishing
- How are messages published to Zeebe (gateway API, broker client, command processors)?
- Trace the path from `PublishMessage` gRPC/REST request through the gateway to the broker.
- Where is the `PublishMessageRequest` defined and how is it transformed into an internal command?
- How does the gateway decide which partition to route the publish-message command to? (This is where hash-based routing on `correlationKey` should appear.)
- Show the actual `PublishMessageProcessor` (or equivalent) on the broker side and what state it writes (MESSAGE.PUBLISHED, etc.).

```
- How is message TTL handled, and how are messages stored in state
(MessageState)?
- How is message deduplication handled via `messageId`?

## 2. Message Subscriptions
Explain the two kinds of subscriptions:
- **Process Message Subscription** (subscription opened by a process
instance waiting on a message catch event / receive task / message
boundary event / message start event).
- **Message Subscription** (the broker-side subscription tracking
which partition is interested in which message name + correlation
key).

For each:
- Where are they opened/closed in the engine?
- What records/intents are involved (e.g. `MESSAGE_SUBSCRIPTION`,
`PROCESS_MESSAGE_SUBSCRIPTION` with intents like CREATING, CREATE,
CREATED, CORRELATING, CORRELATED, DELETING, DELETED)?
- Show the relevant processors (e.g.
`OpenMessageSubscriptionProcessor`,
`CorrelateMessageSubscriptionProcessor`, etc.) with snippets.
- How do subscriptions get sent across partitions
(SubscriptionCommandSender / SubscriptionCommandMessageHandler)?

## 3. Message Correlation
- How are published messages correlated to waiting subscriptions?
- Show the correlation flow when a message is published: lookup of
subscriptions, sending correlate commands cross-partition, ack flow
back.
- Show the correlation flow when a subscription is opened and a
buffered message already exists.
- How are message start events handled (they don't have a process
instance yet)?
- How is the correlation key extracted (FEEL expression evaluation)
at both the publish side and the subscribe side?

## 4. Partitioning & Routing
This is the central piece the user asked about. Explain:
- The hash-based partitioning of messages by `correlationKey`. Where
is this implemented? (Look for something like `SubscriptionUtil`,
`getSubscriptionPartitionId`, `getSubscriptionHashCode`,
`RoundRobinDispatchStrategy`, `RoutingState`,
`MessageCorrelationKeyContext`, etc.)
- Show the actual hash function and modulo-by-partition-count logic.
- Why hash-based: so that the same correlation key always lands on
the same partition, allowing local correlation between
MessageSubscription and Message state.
```

- How does the gateway use this to route `PublishMessage` commands?
- How does the engine on partition A use this to send `OpenMessageSubscription` commands to the correct partition B that owns that correlation key?
- How does Zeebe's dynamic scaling / `RoutingState` (if present) affect this — i.e. how is the number of partitions determined for hashing? Is there a stable `messageCorrelationPartitions` value distinct from total partitions?
- Show snippets from any "routing" related files: `RoutingState`, `RoutingInfo`, `MessageCorrelationPartitions`, `PartitionScalingChecker`, etc.

5. Cross-Partition Communication
- How do partitions talk to each other for subscription open/correlate/reject/close flows?
- Show the `SubscriptionCommandSender` and the corresponding receiver/handler.
- Explain the retry / at-least-once semantics for these inter-partition messages (e.g. `PendingMessageSubscriptionState`, `PendingProcessMessageSubscriptionState` and the checkers that resend pending commands).

6. Message Buffering & Expiry
- How are messages buffered when no subscription exists yet?
- How does TTL expiration work (`MessageTimeToLiveChecker` or similar)?
- How are correlated messages cleaned up?

7. Message Start Events
- Special handling: how are messages correlated to message start events to create a new process instance?
- How does deduplication by `messageId` work for start events?

8. Message Correlation API (newer feature, if present)
- Is there a synchronous "correlate message" command/endpoint that waits for correlation? Show how it works and how it interacts with routing.

Deliverable Format
For each section above, include:
- A clear narrative explanation.
- Concrete code snippets in fenced code blocks with `name=` and `url=` headers pointing to the file at the latest main branch (or a specific commit) on GitHub, with line anchors (e.g. `#L10-L40`).
- A short call-graph or sequence summary (text) where helpful.

Focus especially on the **hash-based routing of messages by

correlation key** and how it ties together publishing, subscriptions, and correlation across partitions — that is the core of the user's question.